```java
package com.codes.java;

import java.util.*;

// ================================================================================
// STRING CODING QUESTIONS
// Questions 1 - 30
// ================================================================================

public class StringCodingQuestionsCode {


    // ================================================================================
    // Question: Reverse String  (Using Two Pointer)
    // Input : "Lawrence"
    // Output: "ecnerwaL"
    // Time Complexity : O(n)
    // Space Complexity: O(n)

    // ================================================================================
    public static String reverseString(String str) {

        if (str == null || str.isEmpty()) return str;

        char[] arr  = str.toCharArray();
        int    left  = 0;
        int    right = arr.length - 1;

        while (left < right) {
            char temp  = arr[left];
            arr[left]  = arr[right];
            arr[right] = temp;
            left++;
            right--;
        }

        return new String(arr);
    }


    // ================================================================================
    // Question: Reverse String Using StringBuilder
    // Input : "Lawrence"
    // Output: "ecnerwaL"
    // Time Complexity : O(n)
    // Space Complexity: O(n)

    // ================================================================================
    public static String reverseStringBuilder(String str) {

        if (str == null) return null;
```

```
48
49          return new StringBuilder(str).reverse().toString();
50      }
51
52
    // ============================================================================
    ==
53      // Question: Palindrome Check
54      // Input : "racecar"
55      // Output: true
56      // (A palindrome reads the same forwards and backwards)
57      // Time Complexity : O(n)
58      // Space Complexity: O(1)
59
    // ============================================================================
    ==
60      public static boolean isPalindrome(String str) {
61
62          if (str == null)
63              return false;
64
65          int left  = 0;
66          int right = str.length() - 1;
67
68          while (left < right) {
69              if (str.charAt(left) != str.charAt(right)){
70                  return false;
71              }
72              left++;
73              right--;
74          }
75
76          return true;
77      }
78
79
    // ============================================================================
    ==
80      // Question: Anagram Check
81      // Input : "silent", "listen"
82      // Output: true
83      // (Two strings are anagrams if they have the same characters, same
    frequency)
84      // Time Complexity : O(n log n)
85      // Space Complexity: O(n)
86
    // ============================================================================
    ==
87      public static boolean isAnagram(String s1, String s2) {
88
89          if (s1 == null || s2 == null){
90              return false;
91          }
92          if (s1.length() != s2.length()){
93              return false;
94          }
95
```

```
 96          char[] a = s1.toCharArray();
 97          char[] b = s2.toCharArray();
 98
 99          Arrays.sort(a);
100          Arrays.sort(b);
101
102          return Arrays.equals(a, b);
103      }
104
105
    // ================================================================
    ==
106      // Question: Group Anagrams
107      // Input : ["eat", "tea", "tan", "ate", "nat", "bat"]
108      // Output: [[eat, tea, ate], [tan, nat], [bat]]
109      // Explanation:
110      // Sort each word and use the sorted word as the key in a HashMap.
111      // All words with the same sorted key are grouped together.
112      public static List<List<String>> groupAnagrams(String[] words) {
113
114          Map<String, List<String>> map = new HashMap<>();
115
116          for (String word : words) {
117
118              char[] chars = word.toCharArray();
119              Arrays.sort(chars);
120
121              String key = new String(chars);
122
123              map.putIfAbsent(key, new ArrayList<>());
124              map.get(key).add(word);
125          }
126
127          return new ArrayList<>(map.values());
128      }
129
130
    // ================================================================
    ==
131      // Question: Character Frequency Count
132      // Input : "banana"
133      // Output: {b=1, a=3, n=2}
134      // Time Complexity : O(n)
135      // Space Complexity: O(n)
136
    // ================================================================
    ==
137      public static Map<Character, Integer> countCharacterFrequency(String str
    ) {
138
139          Map<Character, Integer> map = new LinkedHashMap<>();
140
141          if (str == null) return map;
142
143          for (char ch : str.toCharArray()) {
144              map.put(ch, map.getOrDefault(ch, 0) + 1);
145          }
```

```
146
147         return map;
148     }
149
150
    // ================================================================================
    ==
151     // Question: Count Occurrence Of A Character
152     // Input : "banana", 'a'
153     // Output: 3
154     // Time Complexity : O(n)
155     // Space Complexity: O(1)
156
    // ================================================================================
    ==
157     public static int countOccurrence(String str, char target) {
158
159         if (str == null) return 0;
160
161         int count = 0;
162
163         for (char ch : str.toCharArray()) {
164             if (ch == target){
165                 count++;
166             }
167         }
168
169         return count;
170     }
171
172
    // ================================================================================
    ==
173     // Question: Find Duplicate Characters
174     // Input : "programming"
175     // Output: [r, g, m]
176     // Time Complexity : O(n)
177     // Space Complexity: O(n)
178
    // ================================================================================
    ==
179     public static Set<Character> findDuplicates(String str) {
180
181         Set<Character> set       = new HashSet<>();
182         Set<Character> duplicates = new LinkedHashSet<>();
183
184         if (str == null){
185             return duplicates;
186         }
187
188         for (char ch : str.toCharArray()) {
189             if (!set.add(ch)) {
190                 duplicates.add(ch);
191             }
192         }
193
194         return duplicates;
```

```
195     }
196 
197 
    // ================================================================================
    ==
198     // Question: Remove Duplicate Characters
199     // Input : "programming"
200     // Output: "progamin"
201     // Time Complexity : O(n)
202     // Space Complexity: O(n)
203 
    // ================================================================================
    ==
204     public static String removeDuplicates(String str) {
205 
206         if (str == null || str.isEmpty()){
207             return str;
208         }
209 
210         Set<Character>  set = new LinkedHashSet<>();
211         StringBuilder    sb  = new StringBuilder();
212 
213         for (char ch : str.toCharArray()) {
214             set.add(ch);
215         }
216 
217         for (char ch : set) {
218             sb.append(ch);
219         }
220 
221         return sb.toString();
222     }
223 
224 
    // ================================================================================
    ==
225     // Question: First Non-Repeating Character
226     // Input : "swiss"
227     // Output: 'w'
228     // Time Complexity : O(n)
229     // Space Complexity: O(n)
230 
    // ================================================================================
    ==
231     public static Character firstNonRepeatingCharacter(String str) {
232 
233         if (str == null || str.isEmpty()){
234             return null;
235         }
236 
237         Map<Character, Integer> map = countCharacterFrequency(str);
238 
239         for (char ch : str.toCharArray()) {
240             if (map.get(ch) == 1)
241                 return ch;
242         }
243 
```

```
244         return null;
245     }
246 
247 
    // ================================================================================
    ==
248     // Question: First Repeating Character
249     // Input : "swiss"
250     // Output: 's'
251     // Time Complexity : O(n)
252     // Space Complexity: O(n)
253 
    // ================================================================================
    ==
254     public static Character firstRepeatingCharacter(String str) {
255 
256         if (str == null || str.isEmpty()){
257             return null;
258         }
259 
260         Set<Character> set = new HashSet<>();
261 
262         for (char ch : str.toCharArray()) {
263             if (!set.add(ch))
264                 return ch;
265         }
266 
267         return null;
268     }
269 
270 
    // ================================================================================
    ==
271     // Question: Maximum Occurring Character
272     // Input : "banana"
273     // Output: 'a'  (appears 3 times)
274     // Time Complexity : O(n)
275     // Space Complexity: O(n)
276 
    // ================================================================================
    ==
277     public static Character maxOccurringCharacter(String str) {
278 
279         if (str == null || str.isEmpty()){
280             return null;
281         }
282 
283         Map<Character, Integer> map = countCharacterFrequency(str);
284 
285         Character result = null;
286         int max = 0;
287 
288         for (Map.Entry<Character, Integer> entry : map.entrySet()) {
289             if (entry.getValue() > max) {
290                 max    = entry.getValue();
291                 result = entry.getKey();
292             }
```

```
293         }
294 
295         return result;
296     }
297 
298 
    // ================================================================================
    ==
299     // Question: Isomorphic Strings
300     // Input : s = "egg", t = "add"
301     // Output: true
302     //
303     //    System.out.println(isIsomorphic("foo", "bar"));      // false
304     //    System.out.println(isIsomorphic("paper", "title"));  // true
305     // Explanation:
306     // Check if every character in the first string maps to exactly one
307     // character in the second string and vice versa.
308     //
309     // Time Complexity : O(n)
310     // Space Complexity: O(n)
311 
    // ================================================================================
    ==
312     public static boolean isIsomorphic(String s, String t) {
313 
314         if (s.length() != t.length()) {
315             return false;
316         }
317 
318         Map<Character, Character> map1 = new HashMap<>();
319         Map<Character, Character> map2 = new HashMap<>();
320 
321         for (int i = 0; i < s.length(); i++) {
322 
323             char ch1 = s.charAt(i);
324             char ch2 = t.charAt(i);
325 
326             if (map1.containsKey(ch1)) {
327                 if (map1.get(ch1) != ch2) {
328                     return false;
329                 }
330             } else {
331                 map1.put(ch1, ch2);
332             }
333 
334             if (map2.containsKey(ch2)) {
335                 if (map2.get(ch2) != ch1) {
336                     return false;
337                 }
338             } else {
339                 map2.put(ch2, ch1);
340             }
341         }
342 
343         return true;
344     }
345 
```

```
346
    // ============================================================================
    ==
347     // Question: Minimum Occurring Character
348     // Input : "banana"
349     // Output: 'b'  (appears 1 time)
350     // Time Complexity : O(n)
351     // Space Complexity: O(n)
352
    // ============================================================================
    ==
353     public static Character minOccurringCharacter(String str) {
354
355         if (str == null || str.isEmpty()){
356             return null;
357         }
358
359         Map<Character, Integer> map = countCharacterFrequency(str);
360
361         Character result = null;
362         int min = Integer.MAX_VALUE;
363
364         for (Map.Entry<Character, Integer> entry : map.entrySet()) {
365             if (entry.getValue() < min) {
366                 min    = entry.getValue();
367                 result = entry.getKey();
368             }
369         }
370
371         return result;
372     }
373
374
    // ============================================================================
    ==
375     // Question: Reverse Words In A Sentence
376     // Input : "Java is awesome"
377     // Output: "awesome is Java"
378     // Time Complexity : O(n)
379     // Space Complexity: O(n)
380
    // ============================================================================
    ==
381     public static String reverseWords(String sentence) {
382
383         String[] words = sentence.split(" ");
384
385         List<String> list = Arrays.asList(words);
386
387         Collections.reverse(list);
388
389         return String.join(" ", list);
390     }
391
392
    // ============================================================================
    ==
```

```
393     // Question: Reverse Each Word In A Sentence
394     // Input : "Java Coding"
395     // Output: "avaJ gnidoC"
396     // Time Complexity : O(n)
397     // Space Complexity: O(n)
398 
    // ================================================================================
    ==
399     public static String reverseEachWord(String sentence) {
400 
401         String[] words = sentence.split(" ");
402 
403         List<String> list = Arrays.asList(words);
404 
405         StringBuilder result = new StringBuilder();
406 
407         for (String word : list) {
408             result.append(reverseString(word))
409                     .append(" ");
410         }
411 
412         return result.toString().trim();
413     }
414 
415 
    // ================================================================================
    ==
416     // Question: Find the Longest Common Prefix
417     // Input : ["flower", "flow", "flight"]
418     // Output: "fl"
419 
    // ================================================================================
    ==
420     public static String longestCommonPrefix(String[] strs) {
421 
422         if (strs == null || strs.length == 0) {
423             return "";
424         }
425 
426         String prefix = "";
427 
428         for (int i = 0; i < strs[0].length(); i++) {
429 
430             char ch = strs[0].charAt(i);
431 
432             for (int j = 1; j < strs.length; j++) {
433 
434 //                 if rest other words length is less than first for prefix or
    char is not equal to first word prefix
435                 if (i >= strs[j].length() || strs[j].charAt(i) != ch) {
436                     return prefix;
437                 }
438             }
439 
440             prefix += ch;
441         }
442 
```

```
443         return prefix;
444     }
445 
446 
    // ================================================================
    ==
447     // Question: String Compression
448     // Input : "aabccccccaaa"
449     // Output: "a2b1c5a3"
450     // Time Complexity : O(n)
451     // Space Complexity: O(n)
452 
    // ================================================================
    ==
453     public static String compressString(String str) {
454 
455         if (str == null || str.isEmpty()){
456             return str;
457         }
458 
459         StringBuilder sb = new StringBuilder();
460         int count = 1;
461 
462         for (int i = 0; i < str.length(); i++) {
463 
464             if (i + 1 < str.length() && str.charAt(i) == str.charAt(i + 1)) {
465                 count++;
466             } else {
467                 sb.append(str.charAt(i));
468                 sb.append(count);
469                 count = 1;
470             }
471         }
472 
473         return sb.toString();
474     }
475 
476 
    // ================================================================
    ==
477     // Question: Rotation String Check
478     // Input : "ABCD", "CDAB"
479     // Output: true
480     // (s2 is a rotation of s1 if s2 exists in s1+s1)
481     // Time Complexity : O(n)
482     // Space Complexity: O(n)
483 
    // ================================================================
    ==
484     public static boolean isRotation(String s1, String s2) {
485 
486         if (s1 == null || s2 == null){
487             return false;
488         }
489 
490         return s1.length() == s2.length() && (s1 + s1).contains(s2);
491     }
```

```
492
493
    // ============================================================================
    ==
494     // Question: Check String Contains Only Digits
495     // Input : "12345"
496     // Output: true
497     // Time Complexity : O(n)
498
    // ============================================================================
    ==
499     public static boolean containsOnlyDigits(String str) {
500
501         if (str == null || str.isEmpty()){
502             return false;
503         }
504
505         return str.matches("\\d+");
506     }
507
508
    // ============================================================================
    ==
509     // Question: Remove Whitespaces
510     // Input : "Java Coding Question"
511     // Output: "JavaCodingQuestion"
512     // Time Complexity : O(n)
513
    // ============================================================================
    ==
514     public static String removeWhitespaces(String str) {
515
516         if (str == null) return null;
517
518         return str.replace(" ", "");
519     }
520
521
    // ============================================================================
    ==
522     // Question: Remove Special Characters
523     // Input : "Java@123#Code!"
524     // Output: "Java123Code"
525     // Time Complexity : O(n)
526
    // ============================================================================
    ==
527     public static String removeSpecialCharacters(String str) {
528
529         if (str == null) return null;
530
531         return str.replaceAll("[^a-zA-Z0-9]", "");
532     }
533
534
    // ============================================================================
    ==
```

```java
    // Question: Count Vowels And Consonants
    // Input : "Lawrence"
    // Output: Vowels = 3, Consonants = 5
    // Time Complexity : O(n)

// ================================================================================
==
    public static void countVowelsAndConsonants(String str) {

        int vowels     = 0;
        int consonants = 0;

        if (str == null) {
            System.out.println("    Vowels     : " + vowels);
            System.out.println("    Consonants : " + consonants);
            return;
        }

        str = str.toLowerCase();

        for (char ch : str.toCharArray()) {
            if (Character.isLetter(ch)) {
                if ("aeiou".indexOf(ch) != -1) {
                    vowels++;
                } else {
                    consonants++;
                }
            }
        }

        System.out.println("    Vowels     : " + vowels);
        System.out.println("    Consonants : " + consonants);
    }


// ================================================================================
==
    // Question: CamelCase To SnakeCase
    // Input : "javaCodingQuestion"
    // Output: "java_coding_question"
    // Time Complexity : O(n)

// ================================================================================
==
    public static String camelCaseToSnakeCase(String str) {

        if (str == null || str.isEmpty()) return str;

        return str.replace(" ", "_");
    }


// ================================================================================
==
    // Question: SnakeCase To CamelCase
    // Input : "java_coding_question"
    // Output: "javaCodingQuestion"
```

```java
    // Time Complexity : O(n)

    // ============================================================================
    ==
    public static String snakeCaseToCamelCase(String str) {

        if (str == null || str.isEmpty()) return str;

        StringBuilder result       = new StringBuilder();
        boolean       upperNext    = false;

        for (char ch : str.toCharArray()) {
            if (ch == '_') {
                upperNext = true;
            } else {
                if (upperNext) {
                    result.append(Character.toUpperCase(ch));
                    upperNext = false;
                } else {
                    result.append(ch);
                }
            }
        }

        return result.toString();
    }


    // ============================================================================
    ==
    // Question: Check Two Strings Equal Without equals()
    // Input : "Java", "Java"
    // Output: true
    // Time Complexity : O(n)
    // Space Complexity: O(1)

    // ============================================================================
    ==
    public static boolean areStringsEqual(String s1, String s2) {

        if (s1 == null && s2 == null)
            return true;
        if (s1 == null || s2 == null)
            return false;
        if (s1.length() != s2.length())
            return false;

        for (int i = 0; i < s1.length(); i++) {
            if (s1.charAt(i) != s2.charAt(i))
                return false;
        }

        return true;
    }


    // ============================================================================
```

```
633 ==
634     // Question: Count Number Of Words In A Sentence
635     // Input : "Java Coding Interview"
636     // Output: 3
637     // Time Complexity : O(n)
638 
    // ================================================================
    ==
639     public static int countWords(String sentence) {
640 
641         if (sentence == null || sentence.trim().isEmpty()) return 0;
642 
643         return sentence.trim().split("\\s+").length;
644     }
645 
646 
    // ================================================================
    ==
647     // Question: Longest Palindromic Substring
648     // Input : "babad"
649     // Output: "bab"
650     // Time Complexity : O(n^2)
651     // Space Complexity: O(n)
652 
    // ================================================================
    ==
653     public static String longestPalindromicSubstring(String str) {
654 
655         if (str == null || str.length() < 2) return str;
656 
657         String longest = "";
658 
659         for (int i = 0; i < str.length(); i++) {
660             for (int j = i + 1; j <= str.length(); j++) {
661 
662                 String sub = str.substring(i, j);
663 
664                 if (isPalindrome(sub) && sub.length() > longest.length()) {
665                     longest = sub;
666                 }
667             }
668         }
669 
670         return longest;
671     }
672 
673 
    // ================================================================
    ==
674     // Question: Generate All Substrings
675     // Input : "ABC"
676     // Output: [A, AB, ABC, B, BC, C]
677     // Time Complexity : O(n^2)
678     // Space Complexity: O(n^2)
679 
    // ================================================================
    ==
```

```
680     public static List<String> generateAllSubstrings(String str) {
681 
682         List<String> result = new ArrayList<>();
683 
684         if (str == null) return result;
685 
686         for (int i = 0; i < str.length(); i++) {
687             for (int j = i + 1; j <= str.length(); j++) {
688                 result.add(str.substring(i, j));
689             }
690         }
691 
692         return result;
693     }
694 
695 
    // ============================================================================
    ==
696     // Question: Longest Substring Without Repeating Characters
697     // Input : "abcabcbb"
698     // Output: "abc"
699     // Explanation: "abc" is the longest substring without repeating
    characters.
700 
    // ============================================================================
    ==
701     public static String longestSubstring(String s) {
702 
703         Set<Character> set = new HashSet<>();
704 
705         int left = 0;
706         int maxLength = 0;
707         int start = 0;
708 
709         for (int right = 0; right < s.length(); right++) {
710 
711             // Remove duplicate characters from the left
712             while (set.contains(s.charAt(right))) {
713                 set.remove(s.charAt(left));
714                 left++;
715             }
716 
717             // Add current character
718             set.add(s.charAt(right));
719 
720             // Update longest substring
721             if (right - left + 1 > maxLength) {
722                 maxLength = right - left + 1;
723                 start = left;
724             }
725         }
726 
727         return s.substring(start, start + maxLength);
728     }
729 
730 
    // ============================================================================
```

```
730 ==
731     // Question: Balanced Brackets (Valid Parentheses)
732     // Input : "()[]{}"
733     // Output: true
734     //
735     // Explanation:
736     // Check whether every opening bracket has the correct
737     // closing bracket in the correct order.
738     //
739     // Time Complexity : O(n)
740     // Space Complexity: O(n)
741
    // ================================================================
    ==
742     public static boolean isBalanced(String str) {
743
744         Stack<Character> stack = new Stack<>();
745
746         for (char ch : str.toCharArray()) {
747
748             if (ch == '(' || ch == '{' || ch == '[') {
749                 stack.push(ch);
750             } else {
751
752                 if (stack.isEmpty()) {
753                     return false;
754                 }
755
756                 char top = stack.pop();
757
758                 if ((ch == ')' && top != '(') ||
759                         (ch == '}' && top != '{') ||
760                         (ch == ']' && top != '[')) {
761                     return false;
762                 }
763             }
764         }
765
766         return stack.isEmpty();
767     }
768
769
    // ================================================================
    ==
770     // Question: Balanced Parentheses Check
771     // Input : "({[]})"
772     // Output: true
773     // (Every opening bracket must have a matching closing bracket in correct
     order)
774     // Time Complexity : O(n)
775     // Space Complexity: O(n)
776 //    LIFO Last In First Out
777
    // ================================================================
    ==
778     public static boolean isBalancedParentheses(String str) {
779
```

```
780         if (str == null)
781             return false;
782 
783         Deque<Character> stack = new ArrayDeque<>();
784 
785         for (char ch : str.toCharArray()) {
786 
787             if (ch == '(' || ch == '{' || ch == '[') {
788                 stack.push(ch);
789 
790             } else if (ch == ')' || ch == '}' || ch == ']') {
791 
792 //                str = ")"; for this test case
793                 if (stack.isEmpty())
794                     return false;
795 
796                 char top = stack.pop();
797 
798                 if ((ch == ')' && top != '(')
799                         || (ch == '}' && top != '{')
800                         || (ch == ']' && top != '[')) {
801                     return false;
802                 }
803             }
804         }
805 
806         return stack.isEmpty();
807     }
808 
809 
    // ================================================================================
    ==
810     // Question: Merge Sort
811     // Input : [5, 2, 4, 1, 3]
812     // Output: [1, 2, 3, 4, 5]
813     //
814     // Explanation:
815     // Divide the array into two halves, sort both halves,
816     // then merge them.
817     //
818     // Time Complexity : O(n log n)
819     // Space Complexity: O(n)
820 
    // ================================================================================
    ==
821     //1. Find mid
822     //2. Sort left half
823     //3. Sort right half
824     //4. Merge both halves
825     public static void mergeSort(int[] arr, int left, int right) {
826 
827         if (left >= right)
828             return;
829 
830         int mid = (left + right) / 2;
831 
832         mergeSort(arr, left, mid);
```

```
833         mergeSort(arr, mid + 1, right);
834 
835         merge(arr, left, mid, right);
836     }
837 
838     public static void merge(int[] arr, int left, int mid, int right) {
839 
840         int[] temp = new int[right - left + 1];
841 
842         int i = left;
843         int j = mid + 1;
844         int k = 0;
845 
846         while (i <= mid && j <= right) {
847 
848             if (arr[i] < arr[j]) {
849                 temp[k++] = arr[i++];
850             } else {
851                 temp[k++] = arr[j++];
852             }
853         }
854 
855         while (i <= mid)
856             temp[k++] = arr[i++];
857 
858         while (j <= right)
859             temp[k++] = arr[j++];
860 
861         for (i = 0; i < temp.length; i++) {
862             arr[left + i] = temp[i];
863         }
864     }
865 
866 
    // ================================================================================
    ==
867     // Question: String Permutations
868     // Input : "ABC"
869     // Output: ABC, ACB, BAC, BCA, CAB, CBA
870     // Time Complexity : O(n!)
871 
    // ================================================================================
    ==
872     public static void printPermutations(String str, String permutation) {
873 
874         if (str == null) return;
875 
876         if (str.length() == 0) {
877             System.out.println("    " + permutation);
878             return;
879         }
880 
881         for (int i = 0; i < str.length(); i++) {
882 
883             char   current   = str.charAt(i);
884             String remaining = str.substring(0, i) + str.substring(i + 1);
885 
```

```
886             printPermutations(remaining, permutation + current);
887         }
888     }
889
890
    // ================================================================
    ==
891     //                              RUN ALL METHODS
892
    // ================================================================
    ==
893     public static void run() {
894
895         System.out.println();
896         System.out.println(
    "============================================================");
897         System.out.println("          STRING CODING QUESTIONS - ALL METHODS
                ");
898         System.out.println(
    "============================================================");
899
900         // ------------------------------------------------------------
901         System.out.println("\n1.  Reverse String (Two Pointer)");
902         System.out.println("    Input  : \"Lawrence\"");
903         System.out.println("    Output : " + reverseString("Lawrence"));
904
905         // ------------------------------------------------------------
906         System.out.println("\n2.  Reverse String (StringBuilder)");
907         System.out.println("    Input  : \"Lawrence\"");
908         System.out.println("    Output : " + reverseStringBuilder("Lawrence"
    ));
909
910         // ------------------------------------------------------------
911         System.out.println("\n3.  Palindrome Check");
912         System.out.println("    Input  : \"racecar\"");
913         System.out.println("    Output : " + isPalindrome("racecar"));
914
915         // ------------------------------------------------------------
916         System.out.println("\n4.  Anagram Check");
917         System.out.println("    Input  : \"silent\", \"listen\"");
918         System.out.println("    Output : " + isAnagram("silent", "listen"));
919
920         // ------------------------------------------------------------
921         System.out.println("\n5.  Character Frequency Count");
922         System.out.println("    Input  : \"banana\"");
923         System.out.println("    Output : " + countCharacterFrequency("banana"
    ));
924
925         // ------------------------------------------------------------
926         System.out.println("\n6.  Count Occurrence Of Character");
927         System.out.println("    Input  : \"banana\", 'a'");
928         System.out.println("    Output : " + countOccurrence("banana", 'a'));
929
930         // ------------------------------------------------------------
931         System.out.println("\n7.  Find Duplicate Characters");
932         System.out.println("    Input  : \"programming\"");
933         System.out.println("    Output : " + findDuplicates("programming"));
```

```
934
935         // ------------------------------------------------------------
936         System.out.println("\n8.  Remove Duplicate Characters");
937         System.out.println("    Input  : \"programming\"");
938         System.out.println("    Output : " + removeDuplicates("programming"
    ));
939
940         // ------------------------------------------------------------
941         System.out.println("\n9.  First Non-Repeating Character");
942         System.out.println("    Input  : \"swiss\"");
943         System.out.println("    Output : " + firstNonRepeatingCharacter("
    swiss"));
944
945         // ------------------------------------------------------------
946         System.out.println("\n10. First Repeating Character");
947         System.out.println("    Input  : \"swiss\"");
948         System.out.println("    Output : " + firstRepeatingCharacter("swiss"
    ));
949
950         // ------------------------------------------------------------
951         System.out.println("\n11. Maximum Occurring Character");
952         System.out.println("    Input  : \"banana\"");
953         System.out.println("    Output : " + maxOccurringCharacter("banana"
    ));
954
955         // ------------------------------------------------------------
956         System.out.println("\n12. Minimum Occurring Character");
957         System.out.println("    Input  : \"banana\"");
958         System.out.println("    Output : " + minOccurringCharacter("banana"
    ));
959
960         // ------------------------------------------------------------
961         System.out.println("\n13. Reverse Words In Sentence");
962         System.out.println("    Input  : \"Java is awesome\"");
963         System.out.println("    Output : " + reverseWords("Java is awesome"
    ));
964
965         // ------------------------------------------------------------
966         System.out.println("\n14. Reverse Each Word In Sentence");
967         System.out.println("    Input  : \"Java Coding\"");
968         System.out.println("    Output : " + reverseEachWord("Java Coding"));
969
970         // ------------------------------------------------------------
971         System.out.println("\n15. Longest Common Prefix");
972         System.out.println("    Input  : [\"flower\", \"flow\", \"flight\"]"
    );
973         System.out.println("    Output : " + longestCommonPrefix(
974                 new String[]{"flower", "flow", "flight"}));
975
976         // ------------------------------------------------------------
977         System.out.println("\n16. String Compression");
978         System.out.println("    Input  : \"aabcccccaaa\"");
979         System.out.println("    Output : " + compressString("aabcccccaaa"));
980
981         // ------------------------------------------------------------
982         System.out.println("\n17. Rotation String Check");
983         System.out.println("    Input  : \"ABCD\", \"CDAB\"");
```

```
984         System.out.println("    Output : " + isRotation("ABCD", "CDAB"));
985 
986 
    // ------------------------------------------------------------
987         System.out.println("\n18. Contains Only Digits");
988         System.out.println("    Input  : \"12345\"");
989         System.out.println("    Output : " + containsOnlyDigits("12345"));
990 
991 
    // ------------------------------------------------------------
992         System.out.println("\n19. Remove Whitespaces");
993         System.out.println("    Input  : \"Java Coding Question\"");
994         System.out.println("    Output : " + removeWhitespaces("Java Coding
    Question"));
995 
996 
    // ------------------------------------------------------------
997         System.out.println("\n20. Remove Special Characters");
998         System.out.println("    Input  : \"Java@123#Code!\"");
999         System.out.println("    Output : " + removeSpecialCharacters("Java@
    123#Code!"));
1000 
1001 
    // ------------------------------------------------------------
1002        System.out.println("\n21. Count Vowels And Consonants");
1003        System.out.println("    Input  : \"Lawrence\"");
1004        countVowelsAndConsonants("Lawrence");
1005 
1006 
    // ------------------------------------------------------------
1007        System.out.println("\n22. CamelCase To SnakeCase");
1008        System.out.println("    Input  : \"javaCodingQuestion\"");
1009        System.out.println("    Output : " + camelCaseToSnakeCase("
    javaCodingQuestion"));
1010 
1011 
    // ------------------------------------------------------------
1012        System.out.println("\n23. SnakeCase To CamelCase");
1013        System.out.println("    Input  : \"java_coding_question\"");
1014        System.out.println("    Output : " + snakeCaseToCamelCase("
    java_coding_question"));
1015 
1016 
    // ------------------------------------------------------------
1017        System.out.println("\n24. Check Two Strings Equal (Without equals
    ())");
1018        System.out.println("    Input  : \"Java\", \"Java\"");
1019        System.out.println("    Output : " + areStringsEqual("Java", "Java"
    ));
1020 
1021 
    // ------------------------------------------------------------
1022        System.out.println("\n25. Count Number Of Words");
1023        System.out.println("    Input  : \"Java Coding Interview\"");
1024        System.out.println("    Output : " + countWords("Java Coding
    Interview"));
1025 
```

```
1026
    // ------------------------------------------------------------------
1027        System.out.println("\n26. Longest Palindromic Substring");
1028        System.out.println("    Input  : \"babad\"");
1029        System.out.println("    Output : " + longestPalindromicSubstring("
    babad"));
1030
1031
    // ------------------------------------------------------------------
1032        System.out.println("\n27. Generate All Substrings");
1033        System.out.println("    Input  : \"ABC\"");
1034        System.out.println("    Output : " + generateAllSubstrings("ABC"));
1035
1036
    // ------------------------------------------------------------------
1037        System.out.println("\n28. Longest Substring Without Repeating
    Characters");
1038        System.out.println("    Input  : \"abcabcbb\"");
1039 //        System.out.println("    Output : " +
    longestSubstringWithoutRepeating("abcabcbb"));
1040        System.out.println("    (Longest unique window = \"abc\" = length 3
    )");
1041
1042
    // ------------------------------------------------------------------
1043        System.out.println("\n29. Balanced Parentheses Check");
1044        System.out.println("    Input  : \"({[]})\"");
1045        System.out.println("    Output : " + isBalancedParentheses("({[]})"
    ));
1046
1047
    // ------------------------------------------------------------------
1048        System.out.println("\n30. String Permutations");
1049        System.out.println("    Input  : \"ABC\"");
1050        System.out.println("    Output :");
1051        printPermutations("ABC", "");
1052
1053        System.out.println("\n
    ============================================================");
1054        System.out.println("                        ALL METHODS EXECUTED
     !                    ");
1055        System.out.println(
    "============================================================");
1056        System.out.println();
1057    }
1058
1059 }
```